Tehtävä 1 - Refaktorointi koodin parantamiseksi

Screenshotit alleolevista tiedoista löyty myös repositoriosta, niiden lukeminen oli vaikeaa tässä dokumentissa, joten alla on myös ne käsin kirjoitettuna. Huom. Testiprojekteja ei ollut .zip kansion mukana.

Lähtötilanne

Maintainability Index: 52
Cyclomatic Complexity: 7
Depth of Inheritance: 1
Class Coupling: 2
Lines of Source code: 37
Lines of Executable code: 20

Kaikki on sijoitettu lähinnä Main –metodin sisään, joka siis heikentää ylläpidettävyyttä (Maintainability).

Keskivaihe

Maintainability Index: 69
Cyclomatic Complexity: 8
Depth of Inheritance: 1
Class Coupling: 5
Lines of Source code: 48
Lines of Executable code: 16

Lainasin logiikkaa tehtävänannon mukana tulleesta toisesta projektista, mitä tulee halutun muodon luontiin. Hyödynsin myös Domain –luokkakirjastoa on IShape –rajapinta ja luokat konstruktööreineen niille, joten hyödynsin niitä manuaalisesti luotujen objektien sijaan.

Tämän takia, vaikka koodin määrä nousi, suoritettavan koodin määrä laski. Se myös vaikuttaa ylläpidettyvyyteen, koska vastuuta on siirretty nyt luokille pääprojektista.

Loppuvaihe

Maintainability Index: 84
Cyclomatic Complexity: 8
Depth of Inheritance: 1
Class Coupling: 7
Lines of Source code: 85
Lines of Executable code: 21

Päätin tehdä ns. Wrapper –luokan käyttöliittymään Console –luokalle, joka on parhaillaan tiukasti liitettynä ohjelman toimintaan. Tämä on täysin turha lisäys tämän näin pienessä projektissa, mutta se antaisi mahdollisuuden vaihtaa käyttöliittymän konsolista graafisen-käyttöliittymään ilman ohjelman toiminnan rikkomista.

Se heikentää luettavuutta tämän kokoisessa projektissa, mutta vaihtoehtoisesti se toisi aikaisemman hyödyn sekä mahdollisuuden yksikkötestata projektia, joka muuten olisi todella rasittavaa.

Huomiona, että Maintainability Index nousi keskimääräisesti, mutta esim. Main –metodissa se laski aikaisemmasta 69 -> 61. Tähän on selityksenä se, että luokat luodaan ja hyödynnetään suoraan Main –metodin sisällä. Toisin kuin Dependency Injection -konttien kanssa, joutuisimme manuaalisesti vaihtamaan rajapinnoilla konkreettisen luokan ilmentymän itse luokkaan, mikä vähentää sen ylläpidettävyyttä.

Lisämainintana on, että koko ohjelman toiminta voitaisiin sijoittaa kerroksittain, mikä parantaisi sen pitkäaikaista ylläpidettävyyttä. Sen sijaan, että Main pitäisi sisällään kaiken koodin, sijoittaisimme tietojen kyselyn ja muun omaan luokkaansa, joka hyödyntäisi muita jo luomiamme luokkia ja rajapintoja.

Syy tähän on se, että tämän avulla voisimme luoda ilmentymän tuosta luokasta ja helpottaa meidän testausprosessia. Asioiden testaaminen Main –metodissa on monimutkaista ja hankalaa ja tällä tavoin, voimme siirtää sen paikkaan, jossa sen testaaminen on helpompaa.

Code Metrics -työkalun käyttö tulevaisuudessa

Se on tosi kätevä työkalu ja antaa hyvin vinkkiä, että miten hyvin jokin osa projektia on suunniteltu. Tosin voin sanoa itse, että kun joko luon spagettikoodia tai luen sitä, minulla on jo tunne, että tämä ei oikein ole hyvä juttu ja en pidä tästä.

Tosin se ei ole pahitteeksi käyttää sitä ja katsoa, että mitä mieltä se on koodinlaadusta.